

Learning Experience C.3

Ethan Davidson and Patrick Rooney

ECE 2544

Fall 2024

Section I: Specifications

1. Objective:

- The objective of this Learning Experience is to construct a register transfer system combined with a function unit. This system will comprise four registers and an 8-bit function unit. The registers in this system can receive values from the input switches, send values to the function unit, and receive the output result from the function unit. The function unit in this system can accept two 8-bit two's complement inputs and a 4-bit function select code, and output the 8-bit result, along with four status bits (V, C, N, Z)

2. Requirements and Constraints

- We are only able to edit FunctionUnitandBus.v and waveform files in Quartus.
- If additional modules need to be created they should be implemented in the FunctionUnitandBus.v file.
- Port declarations should not be modified, and neither should anything else that is clearly noted in the comments of the FunctionUnitandBus.v file.
- Function unit must: accept two 8-bit 2's complement operands, accept a 4-bit function select code, and output the result of the operation with four status bits: V (overflow), C (carry), N (negative), Z (zero).
- The function unit must perform seven required operations ($A+B$, $A-B$, A , A' , $B \times 4$, $B \text{ rem } 8$, and either $A \vee B$ or $(A \vee B)'$), the choice of three arithmetic operations, and the choice of three logical or shift operations (13 total operations).
- Design must be implemented with structural and approved dataflow Verilog.
- Design must not create schematics or use behavioral Verilog constructs.
- A unique 4-bit code must be assigned to each operation.
- Efficiency must be considered in the design to minimize FPGA resources.
- Proper coding practices must be used.

Section II: Project Plan

Work Breakdown Structure

Section of Breakdown	Task Number	Task Details
Circuit Design	#1	
	1.1: Truth Table, Kmaps, Expressions	Derived logic expressions for the register and function unit by creating and using truth tables and Kmaps as needed.
	1.2: Register Design	Designed register system based on optimized logic expressions for load and register inputs derived from Kmaps.
	1.3: Function Unit	Fully functional function unit module with operand selection logic, supporting arithmetic, logic shift, and other required blocks are planned out.
Verilog Implementation	#2	
	2.1: Header	Completed file headers with accurate descriptions and version history for all project files.
	2.2:FunctionUnitandBus.v	FunctionUnitandBus.v file containing optimized logic circuit designs with integrated Kmaps and truth tables.
	2.3:Waveform File	Finalized waveform file with all necessary inputs and outputs accurately represented.
	2.4: D-10 lite Testing	Certified that the Verilog implementation on the D-10 lite board works and works correctly.
	2.5:Archived Project Completion	Fully compiled and archived project with finalized circuit, waveform, and headers in place.
Project Report	#3	
	3.1:Constraints and Requirements	Comprehensive understanding of LE specifications and project requirements

		documented and referenced.
	3.2:Project Plan	Our project plan, including WBS and Network diagram.
	3.3:Design Approach	Documented design approach with finalized design decisions based on the completed Quartus project.
	3.4:Simulation	Annotates simulation with operation, source register contents, and destination register contents will be documented
	3.5:Transistor Count and Propagation Delay	Documented transistor count and calculated propagation delays for each module in the Verilog design.
	3.6: Closing Remarks	Insightful summary reflecting on the final outcomes, lessons learned, and overall project takeaways.

Figure 1.1 Learning Experience C.3 Work Breakdown Structure

Network Diagram

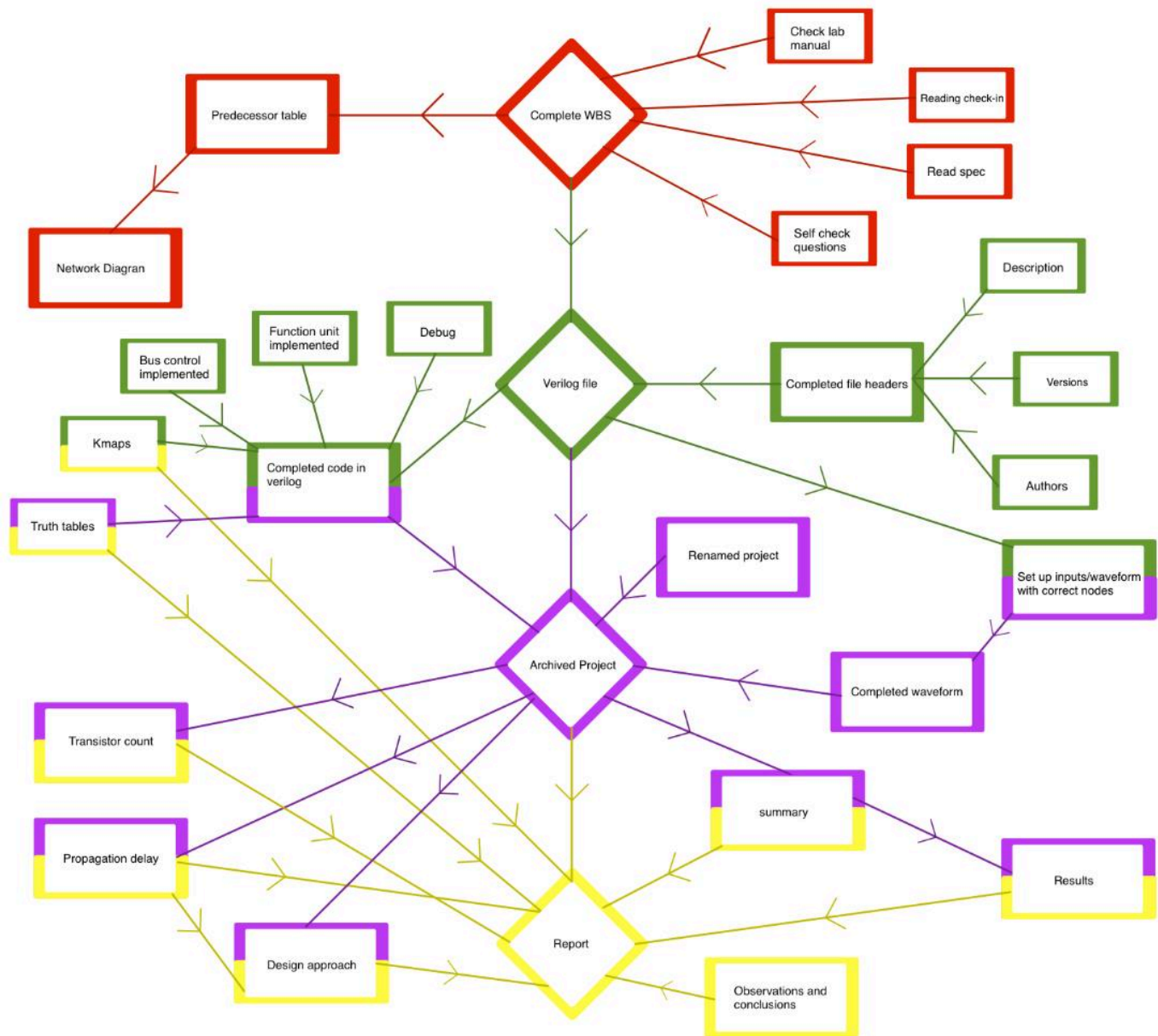


Figure 1.2 Learning Experience C.3 Network Diagram

Section III: Design Approach

Learning Experience C.1a Design

Operation	a input to the adder								b input to the adder								C_0
	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	B_7	B_6	B_5	B_4	B_3	B_2	B_1	B_0	
A + B	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	B_7	B_6	B_5	B_4	B_3	B_2	B_1	B_0	0
A - B	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	B'_7	B'_6	B'_5	B'_4	B'_3	B'_2	B'_1	B'_0	1
A	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	0	0	0	0	0	0	0	0	0
A-1	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	1	1	1	1	1	1	1	1	0
-B	0	0	0	0	0	0	0	0	B'_7	B'_6	B'_5	B'_4	B'_3	B'_2	B'_1	B'_0	1
-A	A'_7	A'_6	A'_5	A'_4	A'_3	A'_2	A'_1	A'_0	0	0	0	0	0	0	0	0	1

Figure 2.1 Selected Operation With Table Inputs For Full Adders C.1a

Select bits			Input bits				Operations
S2	S1	S0	A0	A1	B0	B1	
0	0	0	0	0	0	0	A+B
0	0	1	0	0	0	1	A-B
0	1	0	0	0	1	0	A
0	1	1	x	x	x	x	x
1	0	0	0	0	1	1	A-1
1	0	1	1	0	0	1	-B
1	1	0	x	x	x	x	x
1	1	1	0	1	1	0	-A

Figure 2.2 Select Bits Truth Table C.1a

$A_0 = S_2 S_1$				
A1	$S_0' S_1'$	$S_0' S_1$	$S_0 S_1$	$S_0 S_1'$
S_2'	0	0	x	0
S2	0	x	1	0

Figure 2.3 A_0 Karnaugh Map

$A_1 = S_2 S_1' S_0$				
A0	$S_0' S_1'$	$S_0' S_1$	$S_0 S_1$	$S_0 S_1'$
S_2'	0	0	x	0
S2	0	x	0	1

Figure 2.4 A_1 Karnaugh Map

$B_0 = S_1 + S_2 S_0' S_1'$				
B0	$S_0' S_1'$	$S_0' S_1$	$S_0 S_1$	$S_0 S_1'$
S_2'	0	1	1	0
S2	1	x	1	0

Figure 2.5 B_0 Karnaugh Map

$B_1 = S_2' S_0 + S_2 S_1'$				
B1	$S_0' S_1'$	$S_0' S_1$	$S_0 S_1$	$S_0 S_1'$
S_2'	0	0	x	1
S2	1	x	0	1

Figure 2.6 B_1 Karnaugh Map

Outputs	Expressions
A0	$S_2 S_1$
A1	$S_2 S_1' S_0$
B0	$S_2' S_0 + S_2 S_1'$
B1	$S_1 + S_2 S_0'$

Figure 2.7 Select Bit Logical Expressions

In Learning Experience C.1, we created function blocks that could perform different operations on two 8-bit operands. We created an arithmetic function block in Learning Experience C.1a. Here, we designed a combinational logic circuit that could perform six operations on two 8-bit operand vectors. These operations and their operands are shown in Figure 2.1. In this design we used eight full adders (one for each bit in the operand vectors). We connected two 4x1 muxes (one for the possible values of each operand), and created a combinational circuit for the select bits for these muxes. The truth table for the select bits is shown in Figure 2.2, and the Karnaugh Maps for each select bit (A_0 , A_1 , B_0 , B_1) are shown in Figure 2.3, Figure 2.4, Figure 2.5, and Figure 2.6 respectively. The final minimized Sum of Products (SOP) expressions for each select bit are shown in Figure 2.7

Learning Experience C.1b Design

Operation name	Output	Description
nor	$(A \vee B)'$	Bitwise NOR of A and B
notA	A'	1s complement of A
mult4	$B \times 4$	Multiply B by 4
rem8	$B \text{ rem } 8$	Remainder of B by 8
and	$A \wedge B$	Bitwise AND of A and B
nand	$(A \wedge B)'$	Bitwise NAND of A and B
notB	B'	1s complement of B

Figure 3.1 Seven Selected Operation

Operation	Operation select			Arithmetic Select	Logic select		
	OS[2]	OS[1]	OS[0]	R[0]	T[2]	T[1]	T[0]
nor	0	0	0	x	1	1	1
notA	0	0	1	x	1	0	0
$B \times 4$	1	1	0	0	x	x	x
$B \text{ rem } 8$	1	0	1	1	x	x	x
and	0	1	0	x	0	1	0
nand	0	1	1	x	0	1	1
Not B	1	0	0	x	1	0	0

Figure 3.2 Select Code Of Each Operation

In Learning Experience C.1b, we created a circuit with two function blocks that could also perform seven different operations on two 8-bit operands. These seven operations are shown in Figure 3.1. In this Learning Experience, we created one function block for logical operations and one function block for arithmetic operations. We then created select bits for operation select, arithmetic select, and logic select shown in Figure 3.2.

Learning Experience C.2 Design

Select bits				Status bits				Operations	Block
S3	S2	S1	S0	V	C	N	Z		
0	0	0	0	1	1	1	1	A+B	0
0	0	0	1	1	1	1	1	A-B	
0	0	1	0	1	1	1	1	-A	
0	0	1	1	1	1	1	1	-B	
0	1	0	0	0	0	1	1	A	1
0	1	0	1	0	0	1	1	A'	
0	1	1	0	0	0	1	1	NOR	
0	1	1	1	0	0	1	1	AND	
1	0	0	0	0	0	1	1	NAND	
1	0	0	1	0	0	1	1	B'	
1	0	1	0	0	0	1	1	mult4	2
1	0	1	1	0	0	1	1	Brem8	

Figure 4.1 Selected Operations and Function Blocks

Block 0 = $S_3'S_2'$				
	$S_1'S_0'$	$S_1'S_0$	S_1S_0	S_1S_0'
$S_3'S_2'$	1	1	1	1
$S_3'S_2$	0	0	0	0
S_3S_2	0	0	0	0
S_3S_2'	0	0	0	0

Figure 4.2 Block 0 Select Code Karnaugh Maps

Block 1 = $S_3'S_2 + S_3S_2'S_1'$				
	$S_1'S_0'$	$S_1'S_0$	S_1S_0	S_1S_0'
$S_3'S_2'$	0	0	0	0
$S_3'S_2$	1	1	1	1
S_3S_2	0	0	0	0
S_3S_2'	1	1	0	0

Figure 4.3 Block 1 Select Code Karnaugh Maps

Block 2 = $S_3S_2'S_1$				
	$S_1'S_0'$	$S_1'S_0$	S_1S_0	S_1S_0'
$S_3'S_2'$	0	0	0	0
$S_3'S_2$	0	0	0	0
S_3S_2	0	0	0	0
S_3S_2'	0	0	1	1

Figure 4.3 Block 2 Select Code Karnaugh Maps

In learning Experience C.2, we put everything together and designed a function unit. This function unit performed thirteen different operations on two 8-bit operands. To design this, we broke the function unit up into three function blocks: an arithmetic block, a shift block, and a logic block. Our circuit had four select bit inputs, four status bit outputs (V, C, N, Z), and an 8-bit output of the resulting operation. All of this is shown in a table in *Figure 4.1*. This design consisted of a mux for each function block (three total) that would select a desired operation and a fourth 4x1 mux that would select the desired function block. We created K-Maps to obtain the minimized SOP expressions for each block's select code. The K-Maps, along with their logical expressions for block 0, block 1, and block 2, are shown in *Figure 4.2*, *Figure 4.1*, and *Figure 4.3* respectively.

Function Unit C.3 Design

Select bits				Status bits					
S3	S2	S1	S0	V	C	N	Z	Operations	Block
0	0	0	0	1	1	1	1	A+B	0
0	0	0	1	1	1	1	1	A-B	
0	0	1	0	1	1	1	1	-A	
0	0	1	1	1	1	1	1	-B	
0	1	0	0	1	1	1	1	A-1	
0	1	0	1	0	0	1	1	A	1
0	1	1	0	0	0	1	1	A'	
0	1	1	1	0	0	1	1	NOR	
1	0	0	0	0	0	1	1	AND	
1	0	0	1	0	0	1	1	NAND	
1	0	1	0	0	0	1	1	B'	
1	0	1	1	0	0	1	1	mult4	2
1	1	0	0	0	0	1	1	Brem8	

Figure 5.1 Updated Selected Operations With Function Blocks For The Function Unit For C.3

Block 0 = $S_3'S_2' + S_3'S_1'S_0'$				
	$S_1'S_0'$	$S_1'S_0$	S_1S_0	S_1S_0'
$S_3'S_2'$	1	1	1	1
$S_3'S_2$	1	0	0	0
S_3S_2	0	0	0	0
S_3S_2'	0	0	0	0

Figure 5.2 Block 0 Select Code Karnaugh Map

Block 1 = $S_3'S_2S_0 + S_3S_2'S_1' + S_3'S_2S_1 + S_3S_2'S_0'$				
	$S_1'S_0'$	$S_1'S_0$	S_1S_0	S_1S_0'
$S_3'S_2'$	0	0	0	0
$S_3'S_2$	0	1	1	1
S_3S_2	0	0	0	0
S_3S_2'	1	1	0	1

Figure 5.3 Block 1 Select Code Karnaugh Map

Block 2 = $S_3S_2'S_1S_0 + S_3S_2S_1'S_0'$				
	$S_1'S_0'$	$S_1'S_0$	S_1S_0	S_1S_0'
$S_3'S_2'$	0	0	0	0
$S_3'S_2$	0	0	0	0
S_3S_2	1	0	0	0
S_3S_2'	0	0	1	0

Figure 5.4 Block 2 Select Code Karnaugh Map

Block Select Codes	
Block 0 = $S_3'S_2' + S_3'S_1'S_0'$	
Block 1 = $S_3'S_2S_0 + S_3S_2'S_1' + S_3'S_2S_1 + S_3S_2'S_0'$	
Block 2 = $S_3S_2'S_1S_0 + S_3S_2S_1'S_0'$	

Figure 5.5 Minimized SOP Block Select Codes

When designing the function unit for Learning Experience C.3, we decided to make adjustments to our function unit from Learning Experience C.2. When reviewing our design for the function unit in Learning Experience C.2, we realized that we only included twelve of the thirteen required functions. We added a thirteenth operation by adding the operation “A-1” to our arithmetic function block. This caused us to change all of our select bit codes from “0100” to “1100” (counting up in binary), which meant we also had to recreate our circuit for our select bits. We ended up doing new Karnaugh Maps (K-Maps) to obtain this new circuit. The K-Map for block 0 is shown in Figure 5.2, the K-Map for block 1 is shown in Figure 5.3, and the K-Map for block 2 is shown in Figure 5.4. We kept the three groupings of function blocks the same, with our arithmetic unit being block 0 in Figure 5.1, our logic unit being block 1 in Figure 5.1, and our shift unit being block 2 in Figure 5.1. We also kept the functions in each function block the same. The only changes we made were changing select codes and adding “A-1” to the arithmetic block.

Bus System D.1 Design

Learning Experience D.1 involved implementing a register system that was capable of being a source and destination register based off of certain switch inputs by a DE10 Lite board. The implementation involved a combination of multiplexers, and a decoder that was capable of selecting a register based off of certain inputs, and choosing the load system. Multiplexers were used to choose which register would be used at a time, while a decoder told the register when to load a signal to set it as a destination register. This register system worked only by loading input values from the user into a register, and then moving the value between source and destination registers depending on where the destination and source location was set. We initially chose to use a design that involved more multiplexers for the load of the register, but we quickly realized how we could implement logic gates to get the same output, but in a much more efficient way.

Bus System C.3 Design

For this Learning Experience, we mostly reused the register system from D.1. We chose to reuse the design as the design involved muxes and logic gates which we knew how to debug and reprogram considerably more than a tri state gate bus system. In the overall design of the circuit we considered using a tri state gate multiple times so that we could have the option of a value to not contribute depending on a condition (enable bit). However, we ultimately designed a way around this system and didn't need to implement it for its functionality. The bus system in C.3 was slightly tweaked from D.1 so that it could be used as a source for a function unit, and as a destination from the function unit. This involved changing the decoder to allow keys 1 and 0 to allow loading into registers, and changing a few of the logic gates to be more efficient for the intended design. We also had to change the multiplexers so that their outputs would correspond to operand A, and operand B, which lines up for the 8 bit inputs to the function unit.

Circuits, Models, Discussion

For our final circuit, we made modifications to the function unit from Learning Experience C.2 and we made modifications to the bus system from Learning Experience D.1 (both described in detail above) and combined them to make a function unit and bus system. To make both designs compatible with each other, we had to set the registers to be the inputs of the function unit. This allows the function unit to properly load values from the bus system. We also needed to make modifications to the system so data could be stored in the register system. We did this by using a 4x1 mux to select which register would be assigned to Operand A and Operand B for each operation. We then used a decoder to control when each register would load values, giving us the ability to control which values could be placed in specific registers. To implement our design, we used structural and permitted dataflow in our Verilog model. Another design approach we considered was modifying our bus system from Learning Experience D.2, rather than modifying our bus system from Learning Experience D.1. We ultimately decided against this, since Learning Experience D.2 uses tri state gates as the bus system design, and Learning Experience D.1 uses multiplexers as the bus system design. We decided to modify the bus system from Learning Experience D.1 because we have more experience working with multiplexers and felt more comfortable making a design that could meet the specifications of this project this way.

-

Section IV: Simulation and Testing Results

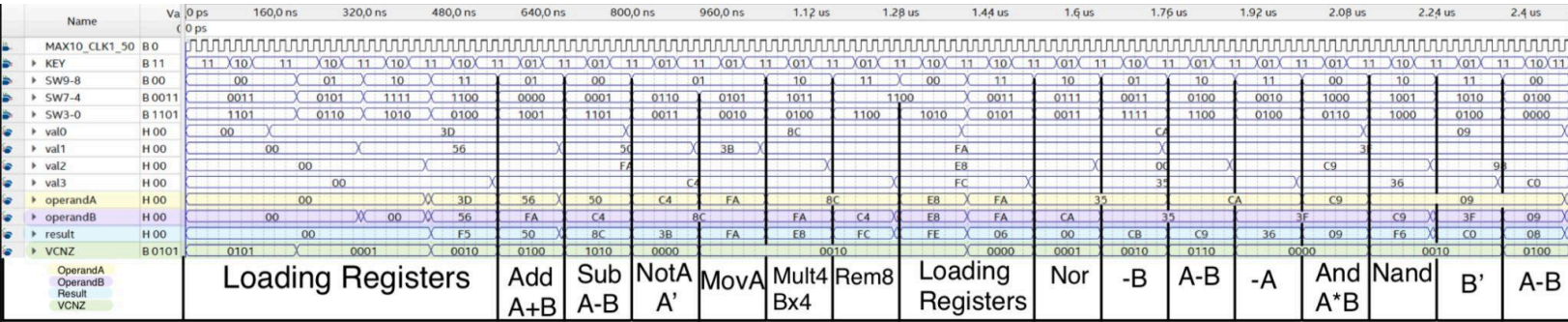


Figure 5.6 Annotated Simulation

Section V: Evaluation of Implementation

Transistor Count

Function Unit Circuit								
Gates / Elements	2AND	3AND	2OR	4x1 Mux	2x1 Mux	NOT	3OR	4AND
Number of Gates	1	5	2	1	3	4	1	2
Transistor Count	6	40	12	62	60	8	8	20
Total Transistors	216							

Figure 6.1 Function Unit Block Select Circuit Transistor Count

Arithmetic Block Full Circuit			
Gates / Elements	XOR	4x1 Mux	Full Adder
Number of Gates	1	18	8
Transistor Count	12	1116	576
Total Transistors	1704		

Figure 6.2 Arithmetic Block Transistor Count

Logic Block Full Circuit					
Gates/ Elements	8x1 mux	NOT	NOR	AND	NAND
Number of Gates	1	2	1	1	1
Transistor Count	104	4	4	6	4
Total Transistors	122				

It is important to note that we had previously calculated that a 2x1 mux has 20 transistors, a 4x1 mux has 62 transistors, and a 8x1 mux has 104 transistors. These ‘assumptions’ are shown in our calculations of final transistor counts. *Figure 6.3 Logic Block Transistor Count*

Shift Block Full Circuit	
Gates/Elements	2x1 Mux
Number of Gates	1
Transistor Count	20
Total Transistors	20

Figure 6.4 Shift Block Transistor Count

Busy System Transistor Count				
Circuit Element	3AND	2OR	NOT	4x1 mux
Number Of Elements	4	4	2	2
Transistor Count	32	24	8	124
Total Transistors	188			

Figure 6.5 Bus System Transistor Count

Total Circuit Transistor Count					
Circuit Element	Function Unit	Arithmetic Block	Logic Block	Shift Block	Bus System
Number Of Elements	1	1	1	1	1
Transistor Count	216	1704	122	20	188
Total Transistors	2250				

Figure 6.6 Total Circuit Transistor Count

As mentioned previously, our final circuit consisted of a circuit for the block select code, an arithmetic block, a logic block, a shift block, and a bus system. Our transistor counts for the block select code (*Figure 6.1*), arithmetic block (*Figure 6.2*), logic block (*Figure 6.3*), the shift block (*Figure 6.4*), and the bus system (*Figure 6.5*) were all added together to find the total transistor count shown in *Figure 6.6*. Due to our modifications to the function unit from Learning Experience C.2, our transistor count increased from 1946 transistors to 2126 transistors. Although this increases the cost of our design, we can justify it by adding an additional function to the arithmetic unit. In our design from Learning Experience C.2, our function unit could only perform twelve operations, however, the design specification requires the function unit to perform thirteen operations. So our increase in transistor count and cost is justified because our modifications ensure that our design meets the requirements outlined by the specification.

Propagation Delay

Total Circuit Propagation Delay						
Gates/ Elements	NOT	4x1 Mux	Full Adder To Carry	Full Adder to Sum	2OR	3AND
Number of Gates	2	3	7	1	1	1
Propagation Delay (ps)	50	450	1225	250	75	100
Total Propagation Delay (ps)	2150					

Figure 7.1 Total Circuit Propagation Delay

We determined that the critical path of our circuit would have been from one of our operand values via the bus system to an arithmetic block. We previously calculated the propagation delays of a 4x1 mux to be 150 ps, full adder to carry to be 175ps, and full adder to sum to be 250ps. These ‘assumptions’ are shown in our calculation of the total propagation delay in Figure 7.1. Although our modifications increased our transistor count, they did not affect the propagation delay of the function unit, as the transistor count of the critical path did not change. The total circuit propagation delay did increase by 350ps (1800ps to 2150ps), since the critical path of the bus system was calculated to be 350ps, and in this circuit we connected the bus system to the function unit. This is more justification for our design changes. Even though our transistor count increased, our propagation delay / speed efficiency of our function unit design did not change and our design now adequately meets the requirements outlined in the specification.

Section VI: Observations and Conclusions

Results

Our results are consistent with the requirements in the specification. We designed a function unit that accepts two 8-bit 2's complement operands, a 4-bit function select code, and outputs the result of the operation with four status bits: V (overflow), C (carry), N (negative), Z (zero). The function unit also performs the required thirteen operations, ours being: $A+B$, $A-B$, A , $A-1$, $-B$, $-A$, $(A \vee B)'$, A' , $B \times 4$, $B \text{ rem } 8$, A^B , $(A^B)'$, and B' . The design for this function unit was implemented using only permitted Verilog practices, and followed all Verilog guidelines outlined in the specification. We also did an adequate job reducing FPGA resources, which we outlined in our discussion in Section V. These are all of the requirements outlined in the specification, which we have adequately met.

Conclusions

The most significant learning experience we obtained from Learning Experience C.3 is working through an iterative design process and combining smaller, previous designs to create a larger working system. Each of the Learning Experience C's directly built off the previous one, as we started by designing function blocks in Learning Experience C.1a and Learning Experience C.1b, and used these function blocks to design a function unit in Learning Experience C.2. Our design process was iterative in Learning Experience C.2, as we worked through both of our designs from Learning Experience C.1a and Learning Experience C.1b again, and used both of these designs to create the larger working system in the function unit of Learning Experience C.2. Learning Experience D.2 modified the bus system from Learning Experience D.1, using tri-state buffers to design the bus system, rather than multiplexers. This was also an iterative design process. In Learning Experience C.3, we modified our function unit from Learning Experience C.2 and we modified our bus system from Learning Experience D.1, using an iterative process, and combined the modified function unit and modified bus system to make a larger function unit and bus system. This experience with an iterative process was very valuable, as this is a crucial process to general engineering, and electrical/computer engineering practices. This was definitely the most involved iterative process either of us had experienced, and this experience was very valuable. The experience with incorporating smaller designs into a larger system was also very valuable as well, as this is a crucial skill to have in electrical and computer

engineering. The fields of electrical engineering, computer engineering, computer science, software engineering, and so on have lots of overlap, and many systems within these fields also incorporate multiple, smaller aspects of these other fields working together in a larger system. This was also the most involved experience either of us have had in incorporating smaller designs into a larger working system, and was a very valuable experience for us both. Overall, between the iterative design process, and the experience with incorporating smaller designs into a larger system, we had many valuable learning experiences in Learning Experience C.3.